UNIVERSIDAD DEL VALLE DE GUATEMALA

Ingeniera de Software 1 - Sección 30

Lynette García

Plataforma digital para la colaboración interdisciplinaria entre asociaciones estudiantiles universitarias

Angel Gabriel Sanabria Morales - 24725

Saul Esteban Castillo Arenas - 24915

Vernel Josué Hernández Cáceres - 24584

Derek Friedhelm Coronado Chilin - 24732

Samuel Antonio Robledo López - 241282

Guatemala, 2026

Product Backlog (Samuel)

Esta parte del informe presenta de manera ordenada las historias de usuario y las tareas que se completaron durante el desarrollo del incremento en el Sprint 1, documentando la pila del producto. El objetivo de esta sección es ofrecer una visión general del producto y demostrar cómo las funciones implementadas satisfacen el flujo principal de uso de la plataforma. En este caso, la plataforma digital tiene como objetivo permitir que las organizaciones estudiantiles universitarias colaboren entre sí de manera interdisciplinaria, centralizando proyectos, permitiendo su visualización y facilitando que los alumnos se postulen para diferentes roles.

El Product Backlog del sprint se organizó en torno a las historias de usuario HU-0, HU-1, HU-2, HU-3 y HU-5, así como a una historia transversal que respalda al sprint. Las historias que se narran van desde la autenticación de los usuarios hasta la configuración inicial de la infraestructura del proyecto, pasando por el listado de proyectos disponibles, la visualización detallada de un proyecto y la aplicación para desempeñar un rol particular. Cada una de estas historias fue dividida en tareas técnicas y operativas que posibilitaron la implementación del sistema, tanto a nivel de backend como de frontend.

Se han llevado a cabo las siguientes tareas: la definición del backlog en Jira, el diseño de la estructura de páginas en Next.js, la creación del panel de control, el listado y detalle de proyectos, el formulario de postulación y sus validaciones, el desarrollo de la página principal, la integración entre frontend y backend, los estilos generales del sistema, las pruebas completas del flujo y la corrección de errores. Esto evidencia que el sprint abarcó no solamente tareas técnicas relacionadas con la infraestructura, sino también funcionalidades que son visibles y accesibles para el usuario final.

Además, esta sección tiene que complementarse con el User Story Mapping, bien sea integrado de manera directa o vinculado como apéndice, dado que este facilita la visualización del panorama total del producto, la trayectoria principal del usuario y la correlación entre las historias puestas en práctica. Así, el backlog no se manifiesta simplemente como un inventario de tareas completadas, sino como una representación organizada del progreso funcional del producto y de la lógica que orienta su desarrollo.

1.1 Pila del Producto

ID	Historia de Usuario	Descripción	Prioridad
HU-01	**Registro e inicio de sesión**	Como estudiante deseo registrarme e iniciar sesión para acceder a la plataforma.	Alta
HU-02	**Completar perfil académico**	Como estudiante deseo completar mi perfil con carrera, habilidades e intereses.	Alta
HU-03	**Explorar proyectos publicados**	Como estudiante deseo explorar proyectos disponibles para encontrar oportunidades.	Alta
HU-04	**Ver detalle de proyecto**	Como estudiante deseo ver la información completa de un proyecto antes de postularme.	Alta
HU-05	**Postularse a un rol en proyecto**	Como estudiante deseo postularme a un rol específico en un proyecto de mi interés.	Alta
HU-06	**Consultar mis postulaciones**	Como estudiante deseo ver el estado de mis postulaciones para dar seguimiento.	Alta
HU-07	**Crear y publicar proyecto**	Como líder de asociación deseo crear y publicar proyectos para reclutar colaboradores.	Media
HU-08	**Gestionar postulaciones recibidas**	Como líder deseo aceptar o rechazar postulaciones para formar mi equipo.	Media
HU-09	**Gestión de tareas del proyecto**	Como colaborador deseo gestionar tareas asignadas dentro del proyecto.	Media
HU-10	**Subir evidencias de trabajo**	Como colaborador deseo subir evidencias para documentar mi participación.	Baja
HU-11	**Validar horas de participación**	Como coordinador deseo validar las horas reportadas por los estudiantes.	Baja
HU-12	**Notificaciones del sistema**	Como usuario deseo recibir notificaciones sobre eventos relevantes.	Baja

Las funcionalidades completadas en este sprint se marcan con el estado correspondiente en la sección de resultados del sprint (sección 4).

1.2 User Story Mapping

El User Story Mapping completo se encuentra disponible en el siguiente enlace:

https://miro.com/welcomeonboard/ajB0YktMWDY5UEtYTVhxU1dwUUZUdG9pSHowYlV0UDNyR3NvWFd6WEdURDRjTEplOHl4RFBQQ2RWSVRhU1NPR3VjWnhDTDBWWndRSEI1YXMxZVN2ckt

lOUNpZ2pOdmhqcytELzZvVCtsS2YraWxSdFBXK2NrODh4NDlNdExzOUlBd044SHFHaVlWYWk0d3NxeHNmeG9BPT0hdjE=?share_link_id=376874944470

Sprint Backlog

1.1. Pila actual del sorint

ID	Tipo	Tarea	Descripción	SP	Horas	Responsable	Fecha Fin
HU-0	**Historia**	**Configuración e infraestructura del proyecto**	**Como equipo, necesitamos configurar el entorno de trabajo para iniciar la implementación.**	**5**	**-**	**Angel Sanabria**	**29/03**
T-01	Subtarea	Configuración del repositorio y estructura del proyecto	Crear monorepo con estructura para frontend y backend, .gitignore, README y docker-compose.	3	5h	Angel Sanabria	28/03
T-02	Subtarea	Configuración de Jira y backlog del sprint	Crear historias, tareas, asignaciones y tablero en Jira.	2	4h	Angel Sanabria	28/03
T-03	Subtarea	Diseño de modelos de base de datos	Implementar entidades principales del MVP en Prisma: Usuario, Proyecto, Postulación.	5	7h	Angel Sanabria	30/03
T-04	Subtarea	Configuración de conexión a base de datos	Configurar PostgreSQL en Docker, Prisma al backend NestJS, ejecutar migraciones.	3	4h	Angel Sanabria	30/03
T-05	Subtarea	Definición de estructura de endpoints	Establecer estructura base de la API con módulos NestJS.	3	4h	Angel Sanabria	31/03
HU-1	**Historia**	**Acceso a la plataforma mediante autenticación**	**Como estudiante, deseo acceder mediante autenticación para utilizar el sistema.**	**8**	**-**	**Angel Sanabria**	**02/04**
T-06	Subtarea	Implementación de autenticación con JWT	Login funcional con JWT, validación de credenciales, guard de autenticación.	5	7h	Angel Sanabria	01/04
T-07	Subtarea	Desarrollo de Landing Page del sistema	Pantalla inicial con presentación de la plataforma y acceso a login.	3	5h	Samuel Robledo	01/04
T-08	Subtarea	Implementación de estructura de páginas en Next.js	Organizar carpetas del frontend para rutas principales.	2	3h	Samuel Robledo	01/04
T-09	Subtarea	Desarrollo del dashboard principal del usuario	Vista principal post-login con visualización de proyectos.	3	5h	Samuel Robledo	02/04

HU-2	**Historia**	**Visualizar listado de proyectos disponibles**	**Como estudiante, deseo ver proyectos disponibles en un feed centralizado.**	**8**	**-**	**Saul Castillo**	**03/04**
T-10	Subtarea	Implementación de endpoint GET /proyectos	Endpoint que devuelve la lista de proyectos disponibles.	3	4h	Saul Castillo	01/04
T-11	Subtarea	Implementación de endpoint POST /proyectos	Endpoint para insertar nuevos proyectos en la BD.	3	4h	Saul Castillo	02/04
T-12	Subtarea	Desarrollo de la vista de listado de proyectos	Pantalla con proyectos disponibles usando Next.js.	3	5h	Saul Castillo	03/04
T-13	Subtarea	Desarrollo del componente de proyecto	Componente reutilizable para representar cada proyecto.	2	3h	Saul Castillo	02/04
HU-3	**Historia**	**Ver detalle de un proyecto específico**	**Como estudiante, deseo ver la información completa de un proyecto.**	**5**	**-**	**Vernel Hernández**	**03/04**
T-14	Subtarea	Implementación de endpoint GET /proyectos/:id	Endpoint que devuelve información específica de un proyecto.	2	3h	Vernel Hernández	01/04
T-15	Subtarea	Desarrollo de la vista detalle de proyecto	Pantalla con información completa del proyecto.	3	5h	Vernel Hernández	03/04
T-16	Subtarea	Integración de frontend con backend para proyectos	Conectar vistas con endpoints. Datos se cargan desde la BD.	3	4h	Vernel Hernández	03/04
T-17	Subtarea	Implementación de navegación mediante Next.js	Organizar carpetas y rutas para navegación entre listado y detalle.	2	2h	Vernel Hernández	02/04
HU-5	**Historia**	**Postularse a un proyecto**	**Como estudiante, deseo postularme a un proyecto enviando justificación.**	**8**	**-**	**Derek Coronado**	**04/04**
T-18	Subtarea	Implementación de endpoint POST /postulaciones	Endpoint para guardar información de postulación.	3	4h	Derek Coronado	02/04
T-19	Subtarea	Desarrollo del formulario de postulación	Interfaz para ingresar información de postulación.	3	5h	Derek Coronado	03/04
T-20	Subtarea	Integración del formulario con backend	Conectar formulario con endpoint de postulaciones.	2	3h	Derek Coronado	03/04
T-21	Subtarea	Implementación de validaciones en formulario	Validar campos requeridos antes de enviar postulación.	2	3h	Derek Coronado	04/04
HU-T	**Historia**	**Tareas transversales del sprint**	**Tareas de integración, pruebas, presentación y estilos.**	**8**	**-**	**Equipo**	**05/04**
T-22	Subtarea	Implementación de estilos generales del sistema	Aplicar estilos con Tailwind CSS para consistencia visual.	3	4h	Samuel Robledo	04/04

T-23	Subtarea	Pruebas del flujo completo del sistema	Verificar flujo completo: login, dashboard, listado, detalle, postulación.	3	4h	Samuel Robledo	05/04
T-24	Subtarea	Corrección de errores generales	Identificar y solucionar errores durante la integración.	2	3h	Samuel Robledo	05/04
T-25	Subtarea	Preparación de presentación del sprint	Diapositivas del sistema desarrollado y decisiones tomadas.	2	4h	Saul Castillo	05/04
T-26	Subtarea	Elaboración del guion de presentación	Definir orden de exposición y demo del sistema.	1	2h	Derek Coronado	05/04

1.2 Calendario de planificacion del sprint

El siguiente calendario muestra la distribución de tareas por integrante a lo largo de los días del sprint:

Fecha	Angel	Saul	Vernel	Derek	Samuel
28/03 Sáb	T-01, T-02	-	-	-	-
29/03 Dom	T-03	-	-	-	-
30/03 Lun	T-03, T-04	-	-	-	-
31/03 Mar	T-05, T-06	T-10	T-14	-	T-07, T-08
01/04 Mié	T-06	T-10, T-11	T-14, T-17	-	T-07, T-09
02/04 Jue	-	T-12, T-13	T-15, T-17	T-18, T-19	T-09
03/04 Vie	-	T-12	T-15, T-16	T-19, T-20, T-21	T-22
04/04 Sáb	-	T-25, T-26	-	T-21, T-25, T-26	T-22, T-23, T-24
05/04 Dom	Revisión	Revisión	Revisión	Revisión	T-23, T-24, Revisión

Incremento

Vínculo al repositorio:

Repositorio: https://github.com/asanabria-2021067/proyecto-ingenieria-software

El código desarrollado y funcional del primer sprint se encuentra alojado en la rama principal (origin/main) del repositorio indicado. El commit auditado para efectos de esta entrega corresponde a e5a50b4, identificado como el merge del pull request #45 desde la rama develop hacia main, el cual integra el flujo completo de detalle de proyecto y postulaciones desarrollado durante el sprint.

Stack Tecnológico

- NestJS 11
- Prisma 6.19.2
- PostgreSQL 17 (Alpine)
- Next.js 15.1.0
- React 19.0.0
- TypeScript 5.7.0
- JWT + bcrypt
- Tailwind CSS
- Radix UI (Material Design 3)
- Node.js 22.

Estructura del proyecto

```
proyecto-ingenieria-software/
  apps/backend/       → API NestJS + Prisma + PostgreSQL
  apps/frontend/      → Next.js 15 + React 19 + TypeScript
  docker-compose.yml  → Orquestación de servicios
  docker-compose.dev.yml → Configuración de desarrollo con hot-reload
```

Estado del código en la rama principal

El repositorio está organizado como un monorepo que separa de forma clara las aplicaciones de backend y frontend, junto con las carpetas de entregables académicos correspondientes a cada corte del curso. La estructura general comprende los directorios apps/backend (API construida en NestJS con Prisma y PostgreSQL), apps/frontend (aplicación construida en Next.js 15 con React 19 y TypeScript), las carpetas Corte 1, Corte 2, Corte 3, Avances 1, Avances 2 y Scrum para la documentación, así como los archivos de orquestación de Docker en la raíz del proyecto.

El backend se encuentra organizado en diez módulos: applications, auth, catalogs, evidence, notifications, prisma, projects, tasks, users y validation. El esquema de base de datos contiene dieciocho enumeraciones y más de treinta modelos relacionales. El frontend está construido con Next.js 15 utilizando App Router e incluye nueve rutas que cubren la landing page, autenticación, dashboard, listado y detalle de proyectos, formulario de postulación y vista de mis postulaciones.

Documentación de los archivos de Docker

La rama principal incluye los archivos necesarios para levantar el sistema mediante Docker. A continuación se describen los archivos presentes y su propósito.

1. **docker-compose.yml (raíz del proyecto)**

   Define la orquestación completa del sistema. Incluye los servicios postgres (importado desde apps/backend/docker-compose.yml), backend y frontend, asociados al perfil app y conectados mediante la red uvg-network. El backend depende de PostgreSQL mediante healthcheck y el frontend depende del backend, garantizando un orden de arranque correcto.

```
services:
  backend:
    build:
      context: ./apps/backend
      dockerfile: Dockerfile
    container_name: uvg-collab-backend
    profiles: [app]
    ports: ["3001:3001"]
    environment:
      -
DATABASE_URL=postgresql://${DB_USER}:${DB_PASSWORD}@postgres:5432/${DB_N
AME}
      - JWT_SECRET=${JWT_SECRET:-dev-secret-change-me}
    depends_on:
      postgres:
        condition: service_healthy
    networks: [uvg-network]

  frontend:
    build:
      context: ./apps/frontend
      args:
        - NEXT_PUBLIC_API_URL=${NEXT_PUBLIC_API_URL:-http://localhost:3001}
    ports: ["3000:3000"]
    depends_on: [backend]
    networks: [uvg-network]
```

2. **docker-compose.dev.yml**

   Sobreescribe la configuración de producción para habilitar un entorno de desarrollo con hot-reload. Utiliza los archivos Dockerfile.dev, monta volúmenes locales sobre las carpetas de código fuente y ejecuta los comandos npm run start:dev (NestJS) y npm run dev (Next.js). Habilita además WATCHPACK_POLLING para asegurar la detección de cambios en sistemas de archivos lentos.

3. **apps/backend/docker-compose.yml**

   Define los servicios de base de datos para desarrollo local. Incluye un servicio postgres basado en postgres:17-alpine, con healthcheck mediante pg_isready, volumen persistente pgdata y exposición

del puerto 5432. También incluye un servicio pgadmin basado en dpage/pgadmin4:latest, expuesto en el puerto 5050.

```
postgres:
  image: postgres:17-alpine
  container_name: uvg-collab-db
  environment:
    POSTGRES_USER: ${DB_USER:-postgres}
    POSTGRES_PASSWORD: ${DB_PASSWORD:-postgres}
    POSTGRES_DB: ${DB_NAME:-uvg_collab}
  ports: ["${DB_PORT:-5432}:5432"]
  volumes: [pgdata:/var/lib/postgresql/data]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
    interval: 10s
    retries: 5
```

4. **apps/backend/Dockerfile**

   Implementa una construcción multi-stage en tres etapas. La primera (deps) instala dependencias mediante npm ci. La segunda (build) genera el cliente Prisma y compila TypeScript. La tercera (production) copia los artefactos necesarios, crea el usuario no root nestjs, expone el puerto 3001 y ejecuta npx prisma migrate deploy seguido de node dist/main.js.

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM node:22-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate && npm run build

FROM node:22-alpine AS production
WORKDIR /app
RUN addgroup -g 1001 -S nodejs && adduser -S nestjs -u 1001
COPY --from=build --chown=nestjs:nodejs /app/dist ./dist
COPY --from=build --chown=nestjs:nodejs /app/node_modules ./node_modules
COPY --from=build --chown=nestjs:nodejs /app/prisma ./prisma
USER nestjs
EXPOSE 3001
CMD npx prisma migrate deploy && node dist/main.js
```

5. **apps/frontend/Dockerfile**

   También utiliza una construcción multi-stage en tres etapas. Recibe como argumentos NEXT_PUBLIC_API_URL y las variables de Cloudinary, las cuales se inyectan en tiempo de compilación. La etapa de producción utiliza la salida standalone de Next.js, crea el usuario no root nextjs, expone el puerto 3000 y ejecuta node server.js como comando inicial.

Funcionalidades planificadas que se completaron

A continuación se listan las subtareas planificadas en el sprint backlog, agrupadas por historia de usuario. Para cada una se describe cómo fue cumplida en la rama principal del repositorio y se incluye, cuando aplica, un fragmento representativo del código correspondiente.

1. **HU-0: Configuración e infraestructura del proyecto**

- ***T-01 — Configuración del repositorio y estructura del proyecto***

  Se configuró el repositorio en GitHub bajo una arquitectura de monorepo que separa el backend y el frontend en la carpeta apps, y mantiene los entregables académicos en carpetas independientes por corte. Se incluye un archivo .gitignore en la raíz, así como un README con la descripción del proyecto, el equipo de desarrollo y la metodología utilizada.

```
proyecto-ingenieria-software/
├── apps/
│   ├── backend/    # API NestJS + Prisma + PostgreSQL
│   └── frontend/   # Next.js 15 + React 19 + TypeScript
├── Corte 1/  Corte 2/  Corte 3/
├── Avances 1/  Avances 2/
├── Scrum/
├── docker-compose.yml
└── README.md
```

- ***T-02 — Configuración de Jira y backlog del sprint***

  La gestión del sprint se realizó en la plataforma Jira, donde se configuró el backlog del producto, se definieron las historias de usuario y se generaron las subtareas correspondientes. La evidencia de esta configuración se encuentra en la plataforma externa de Atlassian y en los entregables académicos del sprint, fuera del repositorio de código.

- ***T-03 — Diseño de modelos de base de datos***

  Se diseñó el modelo de datos completo en el archivo apps/backend/prisma/schema.prisma, definiendo dieciocho enumeraciones y más de treinta modelos relacionales que cubren entidades como Usuario, PerfilEstudiante, Proyecto, RolProyecto, Postulación, Organización, Carrera, Habilidad, Interés, Hito, Tarea, Evidencia y Certificado.

```
enum EstadoProyecto { BORRADOR  PUBLICADO  EN_PROGRESO  CERRADO }
enum EstadoPostulacion { PENDIENTE  ACEPTADA  RECHAZADA }

model Proyecto {
```

```
  idProyecto          Int            @id @default(autoincrement())
  tituloProyecto      String
  descripcionProyecto String
  tipoProyecto        TipoProyecto
  estadoProyecto      EstadoProyecto  @default(BORRADOR)
  modalidadProyecto   ModalidadProyecto?
  // ... relaciones con roles, organizaciones, hitos, tareas
}

model Postulacion {
  idPostulacion       Int              @id @default(autoincrement())
  idUsuarioPostulante Int
  idRolProyecto       Int
  justificacion       String
  estadoPostulacion   EstadoPostulacion @default(PENDIENTE)
}
```

- ***T-04 — Configuración de conexión a base de datos***

  Se implementó el módulo Prisma del backend mediante un PrismaService que extiende PrismaClient y administra el ciclo de vida de la conexión a través de los hooks onModuleInit y onModuleDestroy. La cadena de conexión se inyecta mediante la variable DATABASE_URL definida en docker-compose.yml.

```
@Injectable()
export class PrismaService extends PrismaClient
  implements OnModuleInit, OnModuleDestroy {
  async onModuleInit()    { await this.$connect(); }
  async onModuleDestroy() { await this.$disconnect(); }
}
```

- ***T-05 — Definición de estructura de endpoints***

  Se definieron los diez controladores del backend siguiendo convenciones REST, organizados por módulo (auth, projects, applications, users, catalogs, evidence, notifications, tasks, validation y prisma). Los endpoints utilizan los decoradores estándar de NestJS y se protegen mediante JwtAuthGuard cuando corresponde.

```
@Controller('proyectos')
export class ProjectsController {
  constructor(private projectsService: ProjectsService) {}

  @Get()
  findAll(@Query('q') q?: string) {
    return this.projectsService.findAll(q);
  }

  @Get(':id')
```

```
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.projectsService.findOne(id);
  }

  @Post()
  @UseGuards(JwtAuthGuard)
  create(@Body() data: CreateProjectDto, @CurrentUser() user) {
    return this.projectsService.create(data, user.userId);
  }
}
```

2. HU-1: Acceso a la plataforma mediante autenticación

- ***T-06 — Implementación de autenticación con JWT***

  Se implementó el módulo auth del backend con los endpoints POST /auth/login y POST /auth/register. El flujo de login verifica las credenciales contra la base de datos, compara la contraseña hasheada con bcrypt y emite un access token firmado mediante JwtService. El registro valida que el correo pertenezca al dominio @uvg.edu.gt, hashea la contraseña y crea el usuario junto con su perfil de estudiante en una transacción.

```
async login(loginDto: LoginDto) {
  const usuario = await this.prisma.usuario.findUnique({
    where: { correo: loginDto.correo },
  });
  if (!usuario) throw new UnauthorizedException('Credenciales invalidas');

  const valida = await bcrypt.compare(loginDto.contrasena, usuario.contrasena);
  if (!valida) throw new UnauthorizedException('Credenciales invalidas');

  const payload = { sub: usuario.idUsuario, correo: usuario.correo };
  return { accessToken: this.jwtService.sign(payload) };
}
```

- ***T-07 — Desarrollo de Landing Page del sistema***

  Se desarrolló la página raíz del sistema en apps/frontend/app/page.tsx, compuesta por ocho componentes ubicados en components/landing: Navbar, HeroSection, FeaturesGrid, HowItWorks, IndividualProjectsSection, OrganizationsSection, CTABanner y Footer. La página presenta el propósito del sistema, sus beneficios y los enlaces a login y registro.

```
export default function HomePage() {
  return (
    <main className="min-h-screen bg-surface text-on-surface">
      <Navbar />
  e    <HeroSection />
      <FeaturesGrid />
      <HowItWorks />
```

```
    <IndividualProjectsSection />
    <OrganizationsSection />
    <CTABanner />
    <Footer />
  </main>
 );
}
```

- ***T-08 — Implementación de estructura de páginas en Next.js***

  Se implementó la estructura de páginas mediante el App Router de Next.js 15. Las rutas se definen por estructura de carpetas dentro de apps/frontend/app e incluyen rutas dinámicas mediante los segmentos [id] y [rolId]. El layout raíz aplica la fuente Poppins y envuelve la aplicación con los providers de React Query.

```
app/
├── page.tsx                    # Landing
├── login/page.tsx
├── registro/page.tsx
├── layout.tsx
├── providers.tsx
└── dashboard/
    ├── page.tsx
    ├── proyectos/
    │   ├── page.tsx
    │   └── [id]/
    │       ├── page.tsx
    │       └── postular/[rolId]/page.tsx
    └── mis-postulaciones/page.tsx
```

- ***T-09 — Desarrollo del dashboard principal del usuario***

  Se desarrolló la página /dashboard que muestra una bienvenida personalizada, tres tarjetas con las estadísticas de horas beca acumuladas (con barra de progreso), horas de extensión y proyectos activos, una sección con los proyectos disponibles, una tabla con el estado de las postulaciones recientes y un panel lateral con el perfil del usuario.

```
const horasBeca = stats?.horasBeca ?? 0;
const horasBecaRequeridas = stats?.horasBecaRequeridas ?? 150;
const progressBeca = horasBecaRequeridas > 0
  ? Math.min(100, Math.round((horasBeca / horasBecaRequeridas) * 100))
  : 0;

useEffect(() => {
  getDashboardStats().then(setStats).catch(() => {});
  searchProjects('').then((p) => setProjects(p.slice(0, 4))).catch(() => {});
}, []);
```

3. **HU-2: Visualizar listado de proyectos disponibles:**

- ***T-10 — Implementación de endpoint GET /proyectos***

  Se implementó el endpoint GET /proyectos en el ProjectsController, el cual retorna los proyectos visibles (estado PUBLICADO o EN_PROGRESO y no eliminados), con soporte para búsqueda por título mediante el parámetro de consulta q, ordenamiento por fecha de creación descendente y un límite de veinte resultados.

```
async findAll(q?: string) {
  return this.prisma.proyecto.findMany({
    where: {
      estadoProyecto: { in: ESTADOS_VISIBLES },
      eliminadoEn: null,
      ...(q && q.trim().length > 0
        ? { tituloProyecto: { contains: q.trim(), mode: 'insensitive' } }
        : {}),
    },
    orderBy: { fechaCreacion: 'desc' },
    take: 20,
  });
}
```

- ***T-11 — Implementación de endpoint POST /proyectos***

  Se implementó el endpoint POST /proyectos protegido mediante JwtAuthGuard. Recibe un CreateProjectDto validado mediante class-validator (título de 5 a 200 caracteres, descripción de mínimo 20 caracteres, tipo de proyecto enumerado y campos opcionales como modalidad, ubicación, fechas e identificadores de organizaciones). El proyecto se crea inicialmente en estado BORRADOR y se asocia al usuario autenticado.

```
export class CreateProjectDto {
  @IsString() @MinLength(5) @MaxLength(200)
  tituloProyecto!: string;

  @IsString() @MinLength(20)
  descripcionProyecto!: string;

  @IsEnum(TipoProyecto)
  tipoProyecto!: TipoProyecto;

  @IsOptional() @IsEnum(ModalidadProyecto)
  modalidadProyecto?: ModalidadProyecto;

  @IsOptional() @IsArray() @IsInt({ each: true })
  organizacionesIds?: number[];
}
```

- ***T-12 — Desarrollo de la vista de listado de proyectos***

Se desarrolló la página /dashboard/proyectos, la cual obtiene los proyectos mediante useQuery, los muestra en un grid responsivo de dos columnas y permite filtrarlos en tiempo real por término de búsqueda (sobre título y descripción) y por tipo de proyecto.

```
const { data: proyectos = [] } = useQuery<ProyectoResumen[]>({
  queryKey: ['proyectos'],
  queryFn: () => apiFetch('/proyectos'),
});

const filtrados = proyectos.filter((p) => {
  const coincideBusqueda = !busqueda
    || p.tituloProyecto.toLowerCase().includes(busqueda.toLowerCase())
    || p.descripcionProyecto.toLowerCase().includes(busqueda.toLowerCase());
  const coincideTipo = !tipoFiltro || p.tipoProyecto === tipoFiltro;
  return coincideBusqueda && coincideTipo;
});
```

- ***T-13 — Desarrollo del componente de proyecto***

  Se desarrolló el componente de tarjeta de proyecto que se utiliza dentro del listado. Cada tarjeta muestra el título, la etiqueta del tipo de proyecto, la organización vinculada, una descripción truncada, hasta tres intereses, la modalidad, el número de roles disponibles y un enlace al detalle del proyecto.

```
<div className="bg-surface-container-lowest rounded-2xl p-6 flex flex-col gap-4">
  <h2>{proyecto.tituloProyecto}</h2>
  <span>{TIPO_LABEL[proyecto.tipoProyecto]}</span>
  <p className="line-clamp-2">{proyecto.descripcionProyecto}</p>
  {proyecto.intereses.slice(0, 3).map(({ interes }) => (
    <span key={interes.nombreInteres}>{interes.nombreInteres}</span>
  ))}
  <Link href={`/dashboard/proyectos/${proyecto.idProyecto}`}>
    Ver Proyecto
  </Link>
</div>
```

4. **HU-3: Ver detalle de un proyecto específico:**

- ***T-14 — Implementación de endpoint GET /proyectos/:id***

  Se implementó el endpoint GET /proyectos/:id, el cual valida el identificador con ParseIntPipe y retorna información completa del proyecto, incluyendo creador, organizaciones vinculadas, intereses, roles disponibles con sus requisitos de habilidades, hitos ordenados y tareas. Si el proyecto no existe o no se encuentra visible, lanza una NotFoundException.

```
async findOne(id: number) {
  const proyecto = await this.prisma.proyecto.findFirst({
    where: {
```

```
    idProyecto: id,
    estadoProyecto: { in: ESTADOS_VISIBLES },
    eliminadoEn: null,
  },
  select: {
    idProyecto: true, tituloProyecto: true, descripcionProyecto: true,
    organizaciones: { select: { organizacion: true } },
    roles: {
      select: {
        idRolProyecto: true, nombreRol: true, cupos: true,
        requisitos: { select: { habilidad: true, nivelMinimo: true } },
      },
    },
    hitos: { orderBy: { orden: 'asc' } },
  },
});
if (!proyecto) throw new NotFoundException(`Proyecto ${id} no encontrado`);
return proyecto;
}
```

- ***T-15 — Desarrollo de la vista detalle de proyecto***

  Se desarrolló la página /dashboard/proyectos/[id], que muestra los datos generales del proyecto (título, tipo, organización, modalidad, ubicación, fechas, descripción, objetivos e intereses) y la sección Roles Disponibles con cada rol expandido, sus requisitos de habilidades (destacando los obligatorios) y un botón de postulación.

```
const { id } = useParams<{ id: string }>();
const { data: proyecto } = useQuery<Proyecto>({
  queryKey: ['proyecto', id],
  queryFn: () => apiFetch(`/proyectos/${id}`),
});

{proyecto?.roles.map((rol) => (
  <div key={rol.idRolProyecto}>
    <h3>{rol.nombreRol}</h3>
    <span>{rol.cupos} {rol.cupos === 1 ? 'cupo' : 'cupos'}</span>
    <Link href={`/dashboard/proyectos/${id}/postular/${rol.idRolProyecto}`}>
      Postularme a este Rol
    </Link>
  </div>
))}
```

- ***T-16 — Integración de frontend con backend para proyectos***

  Se implementó un cliente API tipado en lib/api/client.ts que añade automáticamente el token JWT al encabezado Authorization. La gestión del estado del servidor en el frontend se realiza mediante TanStack Query, lo que proporciona caché automática, manejo de estados de carga y de error, y revalidación en segundo plano.

```
export async function apiFetch<T = unknown>(
  endpoint: string, options?: RequestInit,
): Promise<T> {
  const url = `${API_URL}${endpoint}`;
  const token = getToken();
  const headers: HeadersInit = {
    'Content-Type': 'application/json',
    ...(token && { Authorization: `Bearer ${token}` }),
    ...options?.headers,
  };
  const response = await fetch(url, { ...options, headers });
  if (!response.ok) {
    const errorData = await response.json().catch(() => ({}));
    throw new Error(errorData.message || 'Error en la solicitud');
  }
  return response.json();
}
```

- ***T-17 — Implementación de navegación mediante estructura de Next.js***

  La navegación entre las páginas del listado, el detalle y el formulario de postulación se realiza mediante el componente Link de Next.js y los hooks useParams y useRouter del App Router, con rutas dinámicas que utilizan los segmentos [id] y [rolId].

```
// Listado → Detalle
<Link href={`/dashboard/proyectos/${proyecto.idProyecto}`}>Ver</Link>

// Detalle → Postulación
<Link href={`/dashboard/proyectos/${id}/postular/${rol.idRolProyecto}`}>
  Postularme
</Link>

// Parámetros dinámicos
const { id, rolId } = useParams<{ id: string; rolId: string }>();
```

5. **HU-5: Postularse a un proyecto**

- ***T-18 — Implementación de endpoint POST /postulaciones***

  Se implementó el endpoint POST /postulaciones protegido mediante JwtAuthGuard. La lógica del servicio verifica la existencia del usuario y del rol, valida que el proyecto se encuentre en estado PUBLICADO, impide postulaciones duplicadas para un mismo usuario y rol, y crea la postulación en estado PENDIENTE.

```
async create(dto: CreatePostulacionDto) {
  const rol = await this.prisma.rolProyecto.findUnique({
    where: { idRolProyecto: dto.idRolProyecto },
    include: { proyecto: true },
  });
  if (!rol) throw new NotFoundException('Rol no existe');
```

```
  if (rol.proyecto.estadoProyecto !== 'PUBLICADO') {
    throw new BadRequestException('Solo proyectos PUBLICADOS');
  }

  const existente = await this.prisma.postulacion.findFirst({
    where: {
      idUsuarioPostulante: dto.idUsuarioPostulante,
      idRolProyecto: dto.idRolProyecto,
    },
  });
  if (existente) {
    throw new BadRequestException('Ya te postulaste a este rol');
  }

  return this.prisma.postulacion.create({ data: { ...dto } });
}
```

- ***T-19 — Desarrollo del formulario de postulación***

  Se desarrolló la página /dashboard/proyectos/[id]/postular/[rolId], la cual muestra la descripción del rol seleccionado y un formulario con un campo de justificación, contador de caracteres y mensajes de error en línea. El formulario maneja los estados de envío, error y éxito, y al completarse muestra una pantalla de confirmación con un enlace a Mis postulaciones.

```
const schema = z.object({
  justificacion: z.string()
    .min(40, 'La justificación debe tener al menos 40 caracteres.')
    .max(1000, 'La justificación no puede exceder 1000 caracteres.'),
});

const { register, handleSubmit, watch, formState: { errors } } =
  useForm<FormData>({ resolver: zodResolver(schema) });

const justificacion = watch('justificacion', '');

<textarea {...register('justificacion')} rows={8} />
<span>{justificacion?.length ?? 0} / 1000</span>
```

- ***T-20 — Integración del formulario con el backend***

  La integración del formulario con el backend se realiza mediante una mutación de TanStack Query que envía un POST a /postulaciones. El identificador del usuario se obtiene del token JWT almacenado, y los errores devueltos por el backend (postulación duplicada, sesión expirada, entre otros) se exponen al usuario en la interfaz.

```
const mutation = useMutation({
  mutationFn: (data: FormData) => {
```

```
    const userId = getUserIdFromToken();
    if (!userId) throw new Error('Tu sesión ha expirado.');
    return apiFetch('/postulaciones', {
      method: 'POST',
      body: JSON.stringify({
        idUsuarioPostulante: userId,
        idRolProyecto: Number(rolId),
        justificacion: data.justificacion,
      }),
    });
  },
  onError: (err: unknown) => {
    setErrorMsg((err as { message?: string })?.message
      ?? 'Ocurrió un error al enviar tu postulación.');
  },
});
```

- ***T-21 — Implementación de validaciones en el formulario***

  Las validaciones del formulario se implementaron en ambas capas. En el frontend se utiliza un esquema de Zod en conjunto con React Hook Form, lo que permite validación en tiempo real y mensajes de error descriptivos. En el backend, el CreatePostulacionDto utiliza decoradores de class-validator y un ValidationPipe global con whitelist y forbidNonWhitelisted activos.

```
// Backend DTO
export class CreatePostulacionDto {
  @IsInt() @IsPositive() idUsuarioPostulante!: number;
  @IsInt() @IsPositive() idRolProyecto!: number;
  @IsString() @IsNotEmpty() @MinLength(40) @MaxLength(1000)
  justificacion!: string;
}

// main.ts
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,
  forbidNonWhitelisted: true,
  transform: true,
}));
```

6. **HU-T: Tareas transversales del sprint**

- ***T-22 — Implementación de estilos generales del sistema***

  Se configuró Tailwind CSS junto con un sistema de tokens de Material Design 3 expresados como custom properties CSS, y se incorporaron componentes accesibles de Radix UI. La fuente Poppins se carga desde Google Fonts a través del optimizador de fuentes de Next.js.

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    --primary: 142 76% 36%;
    --primary-foreground: 0 0% 98%;
    --surface: 0 0% 100%;
    --on-surface: 0 0% 3.9%;
    --radius: 0.5rem;
  }
}
```

- ***T-23 — Pruebas del flujo completo del sistema***

  Las pruebas del flujo completo del sistema se realizaron de forma manual durante el desarrollo, utilizando los usuarios definidos en el archivo seed de Prisma. Se ejecutó el recorrido completo: registro, inicio de sesión, navegación al listado de proyectos, visualización del detalle, postulación a un rol y verificación en la vista de Mis postulaciones. Esta actividad no produjo archivos de pruebas automatizadas en el repositorio.

- ***T-24 — Corrección de errores generales***

  Durante el sprint se realizaron correcciones iterativas registradas en el historial de commits, abarcando ajustes ortográficos en el formulario de login, mejoras de estilos en la landing page, refactorizaciones para mejorar la legibilidad del código, mejoras en el manejo de errores de los flujos de login y registro, y la incorporación del campo horasBecaRequeridas en el modelo de datos, los DTOs y la interfaz del dashboard.

```
acf594c feat(profile): add horasBecaRequeridas to CompleteProfileDialog
72b7476 fix(landing): update styling for IndividualProjectsSection
a56e715 fix(login): correct spelling of 'Contraseña'
6dfeffb feat(dashboard): use dynamic horasBecaRequeridas in progress
8221fef feat(auth): enhance login error handling and validation
c82e3e6 refactor: improve code structure for readability
```

- ***T-25 — Preparación de presentación del sprint***

  La preparación de la presentación del sprint se realizó utilizando herramientas externas de colaboración, cuyos enlaces se encuentran referenciados en los archivos link_documento.txt de las carpetas Avances 1 y Avances 2. Esta actividad no genera artefactos directamente en el repositorio de código.

- ***T-26 — Elaboración del guion de la presentación***

  El guion de la presentación se elaboró de manera colaborativa por el equipo de desarrollo en un documento externo, en el cual se definieron las secciones que cada integrante presentaría durante la demostración del incremento. Esta actividad tampoco genera artefactos en el repositorio de código.

  Las funcionalidades descritas anteriormente se encuentran integradas en la rama principal del repositorio y pueden ser ejecutadas mediante los archivos de Docker descritos en la sección 3.3

Resultados del Sprint

Lista de tareas por estado

Todas las tareas planificadas para el Sprint 1 fueron completadas exitosamente. La siguiente tabla presenta el estado final:

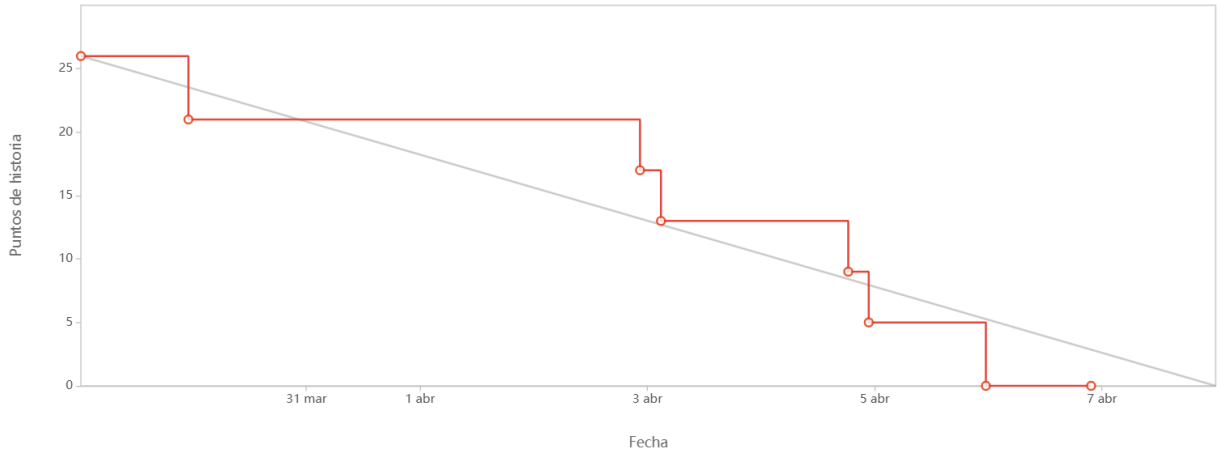
ID	Tarea	Responsable	Estado
HU-0	Configuración e infraestructura del proyecto	Angel Sanabria	**Completada**
T-01	Configuración del repositorio y estructura del proyecto	Angel Sanabria	**Completada**
T-02	Configuración de Jira y backlog del sprint	Angel Sanabria	**Completada**
T-03	Diseño de modelos de base de datos	Angel Sanabria	**Completada**
T-04	Configuración de conexión a base de datos	Angel Sanabria	**Completada**
T-05	Definición de estructura de endpoints	Angel Sanabria	**Completada**
HU-1	Acceso a la plataforma mediante autenticación	Angel Sanabria	**Completada**
T-06	Implementación de autenticación con JWT	Angel Sanabria	**Completada**
T-07	Desarrollo de Landing Page del sistema	Samuel Robledo	**Completada**
T-08	Implementación de estructura de páginas en Next.js	Samuel Robledo	**Completada**
T-09	Desarrollo del dashboard principal del usuario	Samuel Robledo	**Completada**
HU-2	Visualizar listado de proyectos disponibles	Saul Castillo	**Completada**
T-10	Implementación de endpoint GET /proyectos	Saul Castillo	**Completada**

T-11	Implementación de endpoint POST /proyectos	Saul Castillo	**Completada**
T-12	Desarrollo de la vista de listado de proyectos	Saul Castillo	**Completada**
T-13	Desarrollo del componente de proyecto	Saul Castillo	**Completada**
HU-3	Ver detalle de un proyecto específico	Vernel Hernández	**Completada**
T-14	Implementación de endpoint GET /proyectos/:id	Vernel Hernández	**Completada**
T-15	Desarrollo de la vista detalle de proyecto	Vernel Hernández	**Completada**
T-16	Integración de frontend con backend para proyectos	Vernel Hernández	**Completada**
T-17	Implementación de navegación mediante Next.js	Vernel Hernández	**Completada**
HU-5	Postularse a un proyecto	Derek Coronado	**Completada**
T-18	Implementación de endpoint POST /postulaciones	Derek Coronado	**Completada**
T-19	Desarrollo del formulario de postulación	Derek Coronado	**Completada**
T-20	Integración del formulario con backend	Derek Coronado	**Completada**
T-21	Implementación de validaciones en formulario	Derek Coronado	**Completada**
HU-T	Tareas transversales del sprint	Equipo	**Completada**
T-22	Implementación de estilos generales del sistema	Samuel Robledo	**Completada**
T-23	Pruebas del flujo completo del sistema	Samuel Robledo	**Completada**
T-24	Corrección de errores generales	Samuel Robledo	**Completada**
T-25	Preparación de presentación del sprint	Saul Castillo	**Completada**
T-26	Elaboración del guion de presentación	Derek Coronado	**Completada**

- **Tareas en proceso: 0**
  No quedaron tareas en proceso al cierre del sprint.
- **Tareas no concluidas: 0**
  No hubo tareas planificadas que no se pudieran concluir.

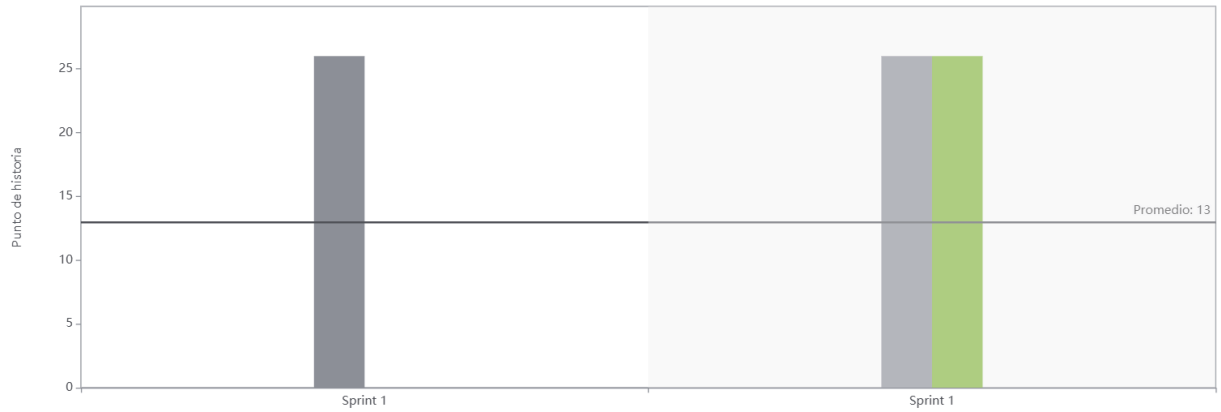
Gráfico Burndown

El siguiente gráfico muestra el trabajo pendiente (en puntos de historia) a lo largo del Sprint 1, comparado con la línea de guía ideal:



Métrica de velocidad

El gráfico de trabajo completado muestra la acumulación de puntos de historia finalizados durante el sprint. Se observa cómo el alcance total del trabajo (línea roja) se mantuvo constante en 26 puntos, mientras que el trabajo completado (línea verde) creció progresivamente, alcanzando el total al cierre del sprint:



(en este caso aparecen dos barras con sprint 1, esto se dio debido a que nosotros realizamos un sprint de prueba, el cual nos ayudó a saber el funcionamiento de Jira, pero el que se debe tomar en cuenta es el que si era el de verdad (que se demuestra con la barra verde, haciendo ver que se completaron todas las tareas teniendo un promedio de 26)

Informe de trabajo completado (BurnUp)

El gráfico de trabajo completado muestra la acumulación de puntos de historia finalizados durante el sprint. Se observa cómo el alcance total del trabajo (línea roja) se mantuvo constante en 26 puntos, mientras que el trabajo completado (línea verde) creció progresivamente, alcanzando el total al cierre del sprint:

Discusión del éxito del sprint basado en las métricas

El Sprint 1 tuvo una duracion aproximada del 31 de marzo al 7 de abril, con un compromiso inicial de 26 puntos de historia. Segun el grafico burndown, se observa que durante los primeros dias del sprint (del 31 de marzo al 2 de abril) el progreso fue minimo, manteniendose la linea real por encima de la linea ideal en aproximadamente 21 puntos. Esto indica que el equipo no comenzo a cerrar tareas de forma significativa sino hasta el 3 de abril, momento a partir del cual el ritmo de trabajo se acelero considerablemente, completando las tareas restantes de forma escalonada hasta llegar a 0 puntos pendientes el 7 de abril. Este patron sugiere una concentracion del esfuerzo en la segunda mitad del sprint.

Respecto a la metrica de velocidad, es importante aclarar que en Jira aparecen dos registros bajo el nombre "Sprint 1": el primero (izquierda) corresponde a un sprint de prueba que el equipo realizo para familiarizarse con la herramienta, por lo que no debe considerarse como dato representativo. El sprint real (derecha) muestra que se comprometieron 26 puntos de historia y se completaron los 26 en su totalidad, lo que representa un 100% de cumplimiento del compromiso. El promedio de velocidad que muestra Jira (13 puntos) esta distorsionado por la inclusion del sprint de prueba y no refleja la capacidad real del equipo.

En conclusion, el Sprint 1 fue exitoso en terminos de alcance: se cumplio con la totalidad de los puntos comprometidos. Sin embargo, el burndown revela una oportunidad de mejora en la distribucion del trabajo a lo largo del sprint, ya que la mayor parte del avance se concentro en los ultimos dias. Para futuros sprints, se recomienda buscar un flujo de trabajo más constante que se aproxime a la linea ideal del burndown.

Evidencias de muestra del incremento a usuarios

Retrospectiva del Sprint

Aspectos positivos

- Cumplimiento total del alcance: Se completaron los 26 puntos de historia comprometidos (100%), logrando entregar un incremento funcional que cubre el flujo principal del sistema: autenticación, exploración de proyectos y postulación.

- Buena división de responsabilidades: Cada integrante tuvo asignaciones claras y diferenciadas. Angel se encargó de la infraestructura y autenticación, Saul del listado de proyectos, Vernel del detalle e integración, Derek de las postulaciones y Samuel de las vistas transversales y estilos. Esto evitó que más de dos personas trabajaran en la misma tarea.

- Stack tecnológico sólido: La decisión de utilizar un monorepo con NestJS, Prisma, PostgreSQL y Next.js permitió un desarrollo organizado y con separación clara de responsabilidades entre frontend y backend.

- Uso efectivo de Docker: La orquestación mediante Docker Compose facilitó que todos los integrantes pudieran levantar el entorno de desarrollo de forma consistente.

- Integración fluida: El uso de TanStack Query y un cliente API centralizado con JWT permitió que la integración frontend-backend fuera limpia y reutilizable.

Aspectos negativos y oportunidades de mejora

- Concentración del trabajo en la segunda mitad del sprint: Como evidencia el burndown, la mayor parte de las tareas se cerraron entre el 3 y 7 de abril, lo que genera riesgo de acumulación y presión al final. Esto se debió en parte a dependencias entre tareas (el frontend dependía de endpoints del backend que se completaron en los primeros días sin cerrar formalmente las historias).

- Sprint de prueba en Jira: La creación de un sprint de prueba antes del sprint real distorsionó la métrica de velocidad promedio. Aunque fue útil para familiarizarse con la herramienta, en el futuro se debe considerar eliminarlo o documentarlo mejor.

- Ausencia de pruebas automatizadas: Las pruebas del flujo se realizaron únicamente de forma manual. Para sprints futuros se recomienda incorporar al menos pruebas unitarias básicas en el backend y pruebas de integración para los endpoints principales.

- Comunicación sobre cambios de estado en Jira: Algunos integrantes no movieron las tareas a "In Progress" al iniciarlas, generando que el tablero no reflejara el estado real del trabajo en tiempo real.

Medidas a tomar para el Sprint 2

1. Cerrar tareas de forma incremental desde el primer día, moviendo las tareas a Done apenas estén funcionalmente completas, para que el burndown refleje el progreso real.
2. Mantener disciplina en el tablero de Jira: cada integrante debe mover sus tareas a "In Progress" al iniciarlas y a "Done" al completarlas.

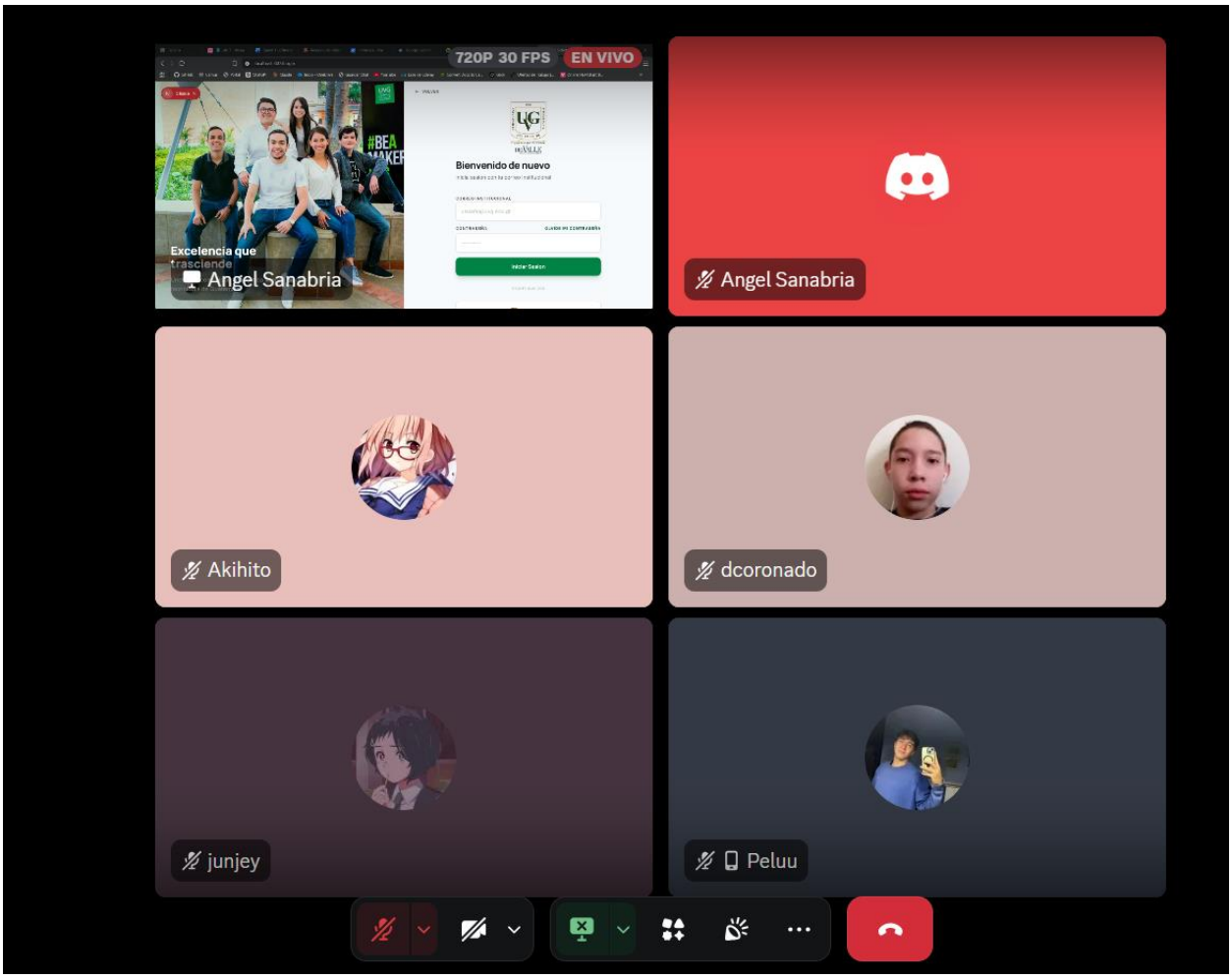
3. Incorporar pruebas automatizadas básicas (al menos para los endpoints principales del backend) como criterio de aceptación.
4. Realizar al menos una sesión de code review formal por cada pull request antes de mergearlo a develop.
5. Distribuir las tareas de integración y pruebas a lo largo del sprint en lugar de concentrarlas al final.

Evidencia de reuniones

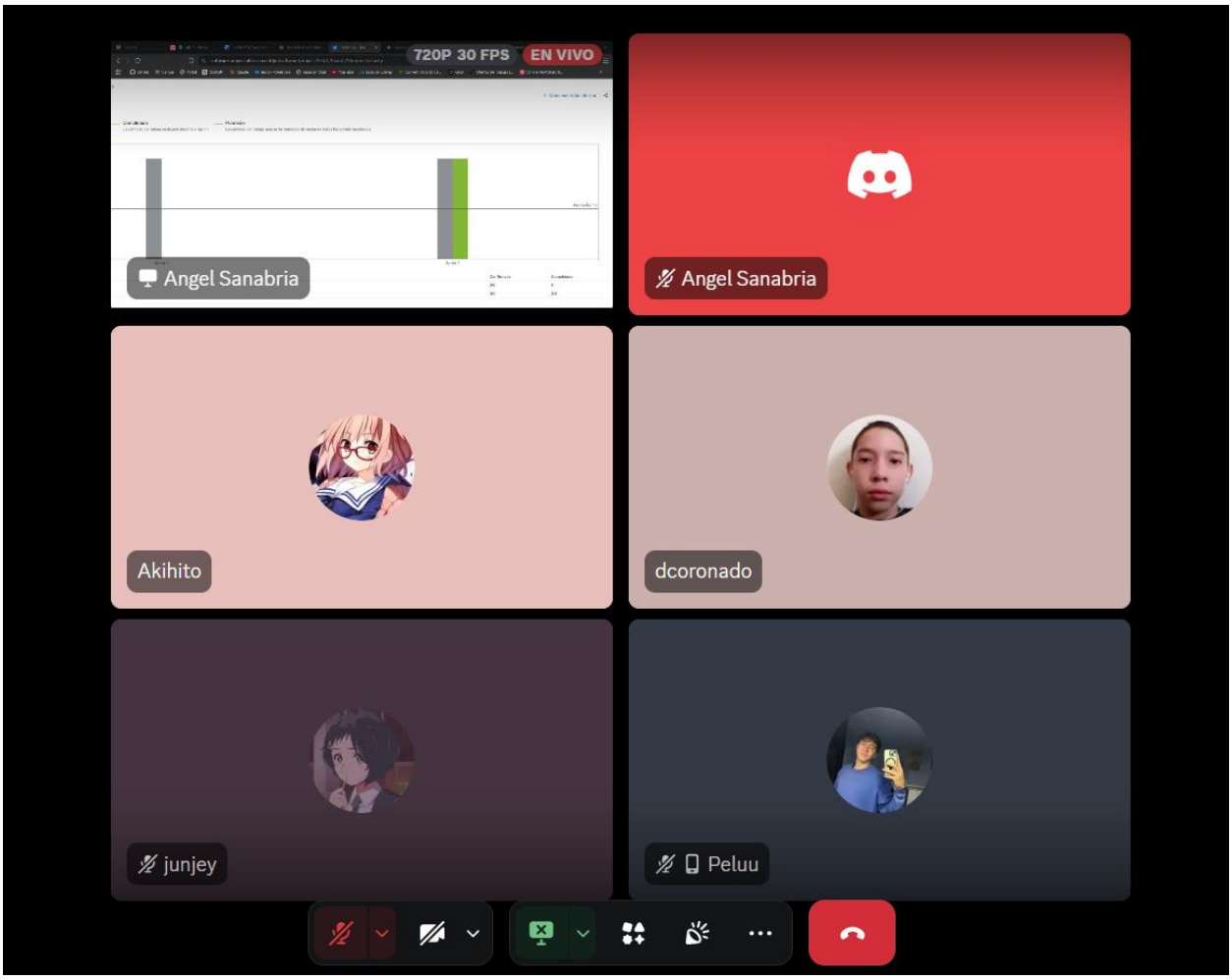
Las reuniones del equipo de desarrollo se realizaron de forma virtual a través de Discord, dado que los horarios y la disponibilidad de los integrantes hacían más práctico el uso de esta plataforma para la comunicación en tiempo real. Se llevaron a cabo las siguientes reuniones durante el Sprint 1:

- **Reunión de Sprint Planning (28 de marzo de 2026):** Se definieron las historias de usuario a incluir en el sprint, se estimaron los story points, se asignaron responsables y se estableció el calendario de planificación. Durante esta reunión se acordó la meta del sprint y se distribuyeron las tareas entre los 5 integrantes del equipo.
- **Reuniones de seguimiento (daily standups):** A lo largo del sprint se realizaron reuniones de seguimiento por Discord en las que cada integrante reportaba el avance de sus tareas, impedimentos encontrados y la planificación para el día siguiente. Estas reuniones se complementaron con comunicación continua a través del canal de texto del servidor de Discord del equipo.
- **Reunión de Sprint Review y Retrospectiva (5 de abril de 2026):** Se revisó el incremento desarrollado, se verificó el cumplimiento de la meta del sprint y se discutieron los aspectos positivos, negativos y las medidas de mejora documentadas en la sección 5 de este informe.

A continuación, se presentan capturas de pantalla de las reuniones realizadas en Discord. No se cuenta con evidencia fotográfica de todas las sesiones debido a que no se estableció como práctica registrar cada reunión desde el inicio del sprint; esto se incorporará como mejora para el Sprint 2.



Esta es una de las reuniones que se hicieron para seguimiento, en este caso yo estaba mostrando los cambios.



Esta fue otra reunión que realizamos esta fue en retrospectiva al Sprint que habiamos realizado.